A1 P2:
Today I’m showing how this program renders a spinning blue cube. It initializes OpenGL, sets up vertex and index data to define the cube’s shape, and sends that data to the GPU using buffers. Shaders handle the transformations and color. In the render loop, a rotation is applied over time, and the cube is redrawn each frame, creating a smooth spinning 3D effect

W4T:
This program simply displays a solid blue screen using OpenGL. After initializing GLFW and GLAD to create the window and load OpenGL functions, it enters the render loop. Inside the loop, the color buffer is cleared each frame using a blue clear color. Since nothing else is drawn, the result is just a plain blue screen displayed continuously until the window is closed.

W4W:

In this program, the triangle is upgraded to display multiple colours.

Each vertex is assigned its own RGB value, and the shaders are updated to pass that colour from the vertex shader to the fragment shader.

Because OpenGL interpolates colours between vertices, the triangle appears with a smooth gradient instead of a single solid colour.

W5T
In this challenge, the goal was to modify Part3-InitOpenGL_3 to display a new background colour while keeping all extended functionality intact.

The program still initializes GLFW and GLEW with an OpenGL 3.3 core profile context. It prints the OpenGL version and renderer information to the console at startup, registers callbacks for the Escape key and framebuffer resizing, and continuously updates FPS and frame time in the window title.

The only functional change is updating the glClearColor values inside the render loop to produce a new window colour.

When running, the window displays the new background colour, shows live performance statistics in the title bar, outputs renderer details in the console, and closes correctly when the Escape key is pressed.”

W6T
In this challenge, the bouncing quad is upgraded with a dynamic transformation matrix and a multi-mode fragment shader.

Instead of simple offsets, a mat4 uniform in the vertex shader now combines rotation and translation, allowing the quad to spin and move via arrow keys simultaneously.

The fragment shader is programmed with three distinct visual modes:

Mode 1 uses a sine wave to pulse the brightness of interpolated vertex colors.

Mode 2 calculates a radial gradient by measuring the distance from the quad's center.

Mode 3 uses procedural math, specifically floor and mod functions to generate a scrolling checkerboard pattern without using any image textures.

By switching between these modes using the keyboard, we demonstrate how uniforms can completely redefine an object's appearance in real-time."

W6W

In this activity, we upgrade our quad from solid colors to Texture Mapping.

Using the stb_image library, we load external images into a custom Texture2D class. We flip the image pixels on load to match OpenGL's bottom-left coordinate system and assign UV coordinates to our vertices to wrap the image perfectly around the quad.

The challenge focuses on Multi-texturing. We bind two different textures an airplane and a crate to separate texture units.

Inside the fragment shader, we use two sampler2D uniforms and the GLSL mix() function to blend the two images together. By applying Mipmaps and linear filtering, we ensure the final result is smooth, detailed, and rendered efficiently in a single pass.

Main

For this project, I've developed a real-time Solar System Simulation using modern OpenGL and C++.

The scene is built on a hierarchical tree structure, allowing planets and moons to orbit their parent bodies with realistic, time-based animation. Each celestial body is rendered using VAOs and VBOs for high-performance memory management on the GPU.

To bring the scene to life, I implemented a Phong Lighting model, calculating ambient, diffuse, and specular components to create realistic shading as the planets revolve around the central sun.

The visual detail is achieved through texture mapping, featuring two distinct types: emissive textures for the sun and standard surface maps for the planets. Finally, the simulation includes an interactive camera system, allowing for real-time navigation and exploration through the 3D workspace. This project demonstrates the effective use of scene hierarchies and lighting in a complex, dynamic environment

Notes:

I’m really sorry but for some reason some videos dont have any sound i tried making them on a different laptop and the issue remained the same even though i just bought this new laptop theres some problems with that so i attached the videos with the sound and i didn’t attach the ones without as it would just be a video of the code only. I also made this document with all the script in case there's any problems with any other videos. Thank you so much.